Campus Shop - Microservices Marketplace

> **Production-grade e-commerce platform** built with microservices architecture on Kubernetes
> **v2.0 Release** - ISO 25010 Compliant | 97% Read Scaling | Event-Driven | Full Observability

Kubernetes PostgreSQL Node.js React Kafka

Table of Contents

Overview

Campus Shop is a **production-ready microservices marketplace** designed for campus communities to buy, sell, and bid on items. Built with modern cloud-native technologies and deployed on **Kind (Kubernetes in Docker)**, it demonstrates enterprise-grade scalability, security, and observability.

Why Campus Shop?

- **Microservices Architecture:** 5 independent services with clear boundaries
- **Kubernetes Orchestration:** 20 pods with autoscaling (HPA/VPA) on Kind cluster
- **Database Replication:** 10 PostgreSQL pods (97% reads from replicas)
- **Event-Driven:** Kafka for async messaging and notifications
- **Full Observability:** Prometheus + Grafana monitoring
- **ISO 25010 Compliant:** Security, reliability, performance, maintainability

---

Features

Core Functionality

Authentication & Security

- Campus email verification (@iitj.ac.in or custom domain)
- JWT-based stateless authentication (24h expiry)
- **Rate limiting:** 5 login attempts/minute per email
- **Password reset:** Time-limited tokens (15min expiry)
- ISO 25010 Security compliance (Confidentiality, Integrity, Authenticity)

Item Management

- **CRUD operations:** Create, read, update, delete items
- **Image uploads:** MinIO S3-compatible storage
- **Server-side pagination:** 12-100 items per page
- **Advanced search:** Case-insensitive search on title + description
- **Price filters:** Min/max price range with sorting (newest, oldest, price)
- Handles 2000+ items efficiently (<250ms response time)

Real-Time Bidding

- Place bids with validation (amount > current highest bid)
- Bid acceptance by item owner
- Active bids tracking
- Bid history per item
- Event-driven notifications via Kafka

Smart Notifications

- **Email alerts:** SMTP integration (Brevo)
- **User preferences:** 3 types (bidReceived, itemSold, bidWon)
- **Notification center:** Bell icon with unread count
- **Mark as read:** Individual or bulk actions
- **MongoDB logs:** Audit trail for all notifications

Profile Management

- Edit displayName and phoneNumber
- Email display (fetched from auth-service)
- **Sold items tab:** Buyer contact info (name, phone, email)
- **Purchased items tab:** Seller contact info + winning bids only
- **Active bids tab:** Track pending bids
- Compact card layout with responsive design

Infrastructure Features

Scalability

- **Horizontal Pod Autoscaling (HPA):** 1-5 replicas per service
- **Database replicas:** Primary + 1 Replica per service (97% read offload)
- **Connection pooling:** Max 10 connections per service (Sequelize)
- **Load tested:** Scales from 2 → 10 pods under k6 load

Monitoring & Observability

- **Prometheus:** Metrics collection from all 5 services
- **Grafana:** Real-time dashboards + alerts (512Mi memory for stability)
- **Custom metrics:** Items created, bids placed, notifications sent
- **Health checks:** `/health` endpoints on all services
- **Resource monitoring:** CPU, memory, database connections
- **Standardized:** All services use consistent resource limits

Event-Driven Architecture

- **Apache Kafka:** Message broker for async communication
- **Topics:** bid.placed, item.sold
- **Consumer:** Notifications service with zero lag
- **Guaranteed delivery:** At-least-once semantics

Architecture

Campus Shop follows a **microservices pattern** with:

- **5 Independent Services:** Auth, Items, Bidding, Notifications, Profile
- **10 PostgreSQL Databases:** Primary + Replica per service
- **API Gateway:** Nginx Ingress Controller
- **Message Broker:** Apache Kafka + Zookeeper
- **Object Storage:** MinIO (S3-compatible)
- **Monitoring:** Prometheus + Grafana

Microservices Breakdown

Service	Port	Database	Responsibilities
Auth	5001	postgres-auth	Registration, login, JWT, rate limiting, password reset
Items	5002	postgres-items	CRUD, image upload, pagination, search, filters
Bidding	5003	postgres-bids	Bid placement, validation, acceptance, Kafka events
Notifications	5004	postgres-notifications + MongoDB	Email sending, preferences, Kafka consumer, logs
Profile	5005	postgres-profiles	User aggregation, sold/purchased items, contact enrichment

---

Technology Stack

Frontend

- **React 18** - Modern UI framework with Hooks
- **Vite** - Fast build tool + HMR

- **Tailwind CSS** - Utility-first styling
- **Axios** - HTTP client
- **React Router** - Client-side routing
- **React Hot Toast** - Toast notifications

Backend

- **Node.js 20** - Runtime environment
- **Express 4** - Web framework
- **Sequelize 6** - PostgreSQL ORM
- **JWT** - Authentication
- **express-rate-limit** - Rate limiting
- **Multer** - File uploads
- **Nodemailer** - Email (SMTP)
- **prom-client** - Prometheus metrics
- **kafkajs** - Kafka client

Databases

- **PostgreSQL 16** - Primary data store (5 databases with replication)
- **MongoDB 7** - Notification logs (polyglot persistence)
- **MinIO** - S3-compatible object storage

Infrastructure

- **Kubernetes 1.29** - Container orchestration
- **Kind (Kubernetes in Docker)** - Local Kubernetes cluster for development
- **Docker** - Containerization
- **Nginx Ingress** - API gateway + routing
- **Apache Kafka 3.x** - Message broker
- **Prometheus 2.x** - Metrics collection
- **Grafana 10.x** - Visualization
- **metrics-server** - K8s resource metrics

Quick Start

Prerequisites

- Docker (24.x or higher)
- Kind (0.20.x or higher)
- kubectl (1.29 or higher)
- Node.js 20+ and npm (for frontend development)

Installation

Full installation guide: See INSTALLATION.md

```
# 1. Clone repository
git clone https://github.com/sujiv1204/campusShop.git
cd campusShop

# 2. Create Kind cluster
kind create cluster --name microservices-cluster --image kindest/node:v1.29.4

# 3. Install cluster components (metrics-server, VPA)
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
# ... (see INSTALLATION.md for VPA setup)

# 4. Deploy secrets and infrastructure
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/secrets/
kubectl apply -f k8s/databases/
kubectl apply -f k8s/kafka/
kubectl apply -f k8s/storage/

# 5. Build and load service images
docker build -t auth-service:local ./services/auth-service
docker build -t items-service:local ./services/items-service
```

```
docker build -t bidding-service:local ./services/bidding-service
docker build -t notifications-service:local ./services/notifications-service
docker build -t profile-service:local ./services/profile-service

kind load docker-image auth-service:local --name microservices-cluster
kind load docker-image items-service:local --name microservices-cluster
kind load docker-image bidding-service:local --name microservices-cluster
kind load docker-image notifications-service:local --name microservices-cluster
kind load docker-image profile-service:local --name microservices-cluster

# 6. Deploy services
kubectl apply -f k8s/deployments/
kubectl apply -f k8s/autoscaling/

# 7. Install Ingress
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml
kubectl apply -f k8s/ingress.yaml

# 8. Port forward (in separate terminals)
kubectl port-forward -n ingress-nginx svc/ingress-nginx-controller 8080:80
kubectl port-forward -n campus-shop svc/minio-service 9000:9000 9001:9001

# 9. Verify health
curl http://localhost:8080/api/auth/health
curl http://localhost:8080/api/items/health
```

Frontend Setup

```
cd frontend
npm install
npm run dev
# Frontend runs at http://localhost:5173
```

Access Points

- **Frontend:** http://localhost:5173
- **API Gateway:** http://localhost:8080/api
- **MinIO Console:** http://localhost:9001 (admin/minioadmin)
- **Prometheus:** http://localhost:9090 (via port-forward)
- **Grafana:** http://localhost:3000 (via port-forward)

Documentation

Document	Description
README.md	This file - project overview
INSTALLATION.md	Complete Kubernetes setup guide
TESTING.md	Testing guide and results
DATABASE_GUIDE.md	Database inspection commands

Performance

System Metrics (Tested: November 2025)

Infrastructure:

- **Pods:** 19 pods across 5 microservices + databases + messaging
- **Database Cluster:** 10 PostgreSQL pods (5 primary + 5 replica) + 1 MongoDB
- **Message Broker:** Kafka + Zookeeper (2 pods)
- **Storage:** MinIO (S3-compatible object storage)
- **Monitoring:** Prometheus + Grafana

Current Scale:

- **Users:** 1,512+ registered users
- **Items:** 4,489+ items (454 sold)

- **Bids:** 7,027+ bids placed
- **Notifications:** 7,203+ notifications generated
- **Kafka Events:** 7,488+ events processed
- **Zero Consumer Lag:** Real-time event processing

Database Performance (CQRS Pattern)

- **Read Replica Usage:** 97%+ reads served by replicas
- **Replication Status:** All replicas streaming (HEALTHY)
- **Replication Lag:** <100ms average (PostgreSQL streaming replication)
- **Connection Pooling:** Max 10 per service (Sequelize)
- **Database Memory:** 22-34Mi per PostgreSQL pod

API Throughput (Load Test Results)

Auth Service:

- User Registration: 1 req/sec sustained throughput
- Load Test: 50 users in 137s (100% success rate)
- Active Users: 1,500+ in database

Items Service:

- Item Creation: **33 req/sec** peak throughput
- Load Test: 100 items in 3s (100% success rate)
- Full CRUD Operations: Create, Read, Update, Delete, Mark as Sold
- Image Upload: MinIO S3 integration working
- Total Items: 4,489+ items

Bidding Service:

- Bid Placement: 9 req/sec average throughput
- Bid Queries: Get all bids, Get bids for item
- Total Bids: 7,027+ bids

Profile Service:

- Profile CRUD operations
- Posted items, Sold items, My bids
- Active bids, Purchased items
- User profile visibility
- Total Profiles: 620+

Notifications Service:

- Kafka event consumption (zero lag)
- Notification queries (paginated)
- Unread counts, Mark as read, Delete
- Notification stats and batch operations
- Processed Events: 7,488+
- MongoDB Notifications: 7,203+

Resource Utilization (All Services)

Microservices:

- Auth: 6m CPU, 59Mi memory
- Items: 41m CPU, 66Mi memory
- Bidding: 48m CPU, 56Mi memory
- Notifications: 51m CPU, 74Mi memory
- Profile: 10m CPU, 50Mi memory

Infrastructure:

- MongoDB: 167m CPU, 326Mi memory
- Kafka: 16m CPU, 442Mi memory
- MinIO: 3m CPU, 150Mi memory
- PostgreSQL (avg): 16m CPU, 27Mi memory per pod

All pods running well within limits (512Mi memory, 1 CPU)

Autoscaling (HPA)

- **Auth Service:** 17% CPU utilization (1/5 pods)
- **Items Service:** 27% CPU utilization (1/5 pods)
- **Bidding Service:** 28% CPU utilization (1/5 pods)
- **Profile Service:** 12% CPU utilization (1/5 pods)
- **Threshold:** Auto-scale at 60% CPU
- **Max Replicas:** 5 per service

Monitoring & Stability

- **Grafana:** Stable operation (512Mi memory)
- **All Services:** Zero failures in system tests
- **Database Replication:** All healthy (streaming status)
- **Kafka Consumer:** Zero lag (real-time processing)
- **Uptime:** 3+ days stable operation

Security & Quality

ISO 25010 Compliance

Security

- **Confidentiality:** JWT tokens, password hashing (bcrypt)
- **Integrity:** Database constraints, ACID transactions
- **Authenticity:** Email verification, JWT signatures
- **Accountability:** Login tracking, notification logs
- **Non-repudiation:** Email audit trail, timestamps

Reliability

- **Availability:** Database replication, HPA, DoS protection
- **Fault Tolerance:** Kafka retries, replica failover
- **Recoverability:** Persistent storage, event replay

Performance Efficiency

- **Time Behavior:** <500ms for 95% of requests
- **Resource Utilization:** CPU limits (1 core max), memory limits (512Mi max)
- **Capacity:** Handles 2000+ items, 5000+ bids, 500+ users

Maintainability

- **Modularity:** Clear service boundaries, database per service
- **Reusability:** Middleware, Sequelize models, React components
- **Testability:** Health endpoints, test scripts, 81.25% coverage

Portability

- **Adaptability:** Environment-based config via K8s Secrets
- **Installability:** One-command deployment (`kubectl apply`)
- **Cloud Ready:** Can deploy to GKE, EKS, AKS

Testing

Test Coverage

Overall: 26/32 endpoints tested (81.25% coverage)

Service	Tested	Total	Coverage	Success Rate
Auth	1	6	16.67%	100%
Items	8	8	**100%**	100%
Bidding	3	3	**100%**	100%
Notifications	6	6	**100%**	100%
Profile	8	9	88.89%	100%

Tested Endpoints:

Auth Service (1/6):

- POST /api/auth/register

Items Service (8/8 - 100%):

- POST /api/items (Create item)
- GET /api/items (Get all items)
- GET /api/items/:id (Get item by ID)
- PUT /api/items/:id (Update item)
- POST /api/items/:id/sell (Mark as sold)
- DELETE /api/items/:id (Delete item)
- POST /api/items/:id/image (Upload image)
- GET /api/items/me/bids (Get bids on my items)

Bidding Service (3/3 - 100%):

- POST /api/bids (Place bid)
- GET /api/bids (Get all bids)
- GET /api/bids/item/:itemId (Get bids for item)

Notifications Service (6/6 - 100%):

- GET /api/notifications (Get notifications with pagination)
- GET /api/notifications/unread-count (Get unread count)
- GET /api/notifications/stats (Get notification stats)
- PATCH /api/notifications/:id/read (Mark as read)
- PATCH /api/notifications/mark-all-read (Mark all as read)
- DELETE /api/notifications/:id (Delete notification)

Profile Service (8/9):

- GET /api/profiles/me (Get my profile)
- GET /api/profiles/:userId (Get other user profile)
- PUT /api/profiles/me (Update profile)
- GET /api/profiles/me/items/posted (Get posted items)
- GET /api/profiles/me/items/sold (Get sold items)
- GET /api/profiles/me/items/purchased (Get purchased items)
- GET /api/profiles/me/bids (Get my bids)
- GET /api/profiles/me/bids/active (Get active bids)

Result: 4 services at 100% coverage | 100% success rate on all tested endpoints

Run Tests

```
cd scripts

# Quick test (5 users, 10 items, 5 bids)
./test-system.sh --quick

# Medium test (200 users, 500 items, 200 bids)
./test-system.sh --medium

# Large test (500 users, 2000 items, 1000 bids)
./test-system.sh --large

# Test specific components
./test-system.sh --auth           # Auth service only
./test-system.sh --items          # Items service only
./test-system.sh --bidding        # Bidding service only
./test-system.sh --db             # Database replicas
./test-system.sh --kafka          # Kafka pipeline

# Custom log file
./test-system.sh --large --log my-test.log
```

Testing Guide: See TESTING.md for detailed testing documentation

---

License

MIT License - See LICENSE file for details